Lógica para Engenharia de Software

Álgebra Booleana

Professor: Me. Sergio Souza Novak
Faculdade de Engenharia de Software



Sumário

1. 1. Introdução e Conceitos Fundamentais

2. 4. Aplicações Práticas (Hardware e Software)
   - 4.1. Hardware e Circuitos Digitais

3. 3. Técnicas de Simplificação Booleana
   - 4.2. Lógica de Programação e Software

4. 2. Propriedades e Teoremas Fundamentais

5. 5. Exercícios de Prática e Fixação

O que é a Álgebra Booleana?

Definição

A **Álgebra Booleana** é um sistema algébrico que opera sobre apenas dois valores: **0** (falso) e **1** (verdadeiro).

Operações

AND $\cdot$ OR $+$ NOT $^{-}$

0 | **1**

Falso Verdadeiro

Contexto Histórico

- Criada por **George Boole** em 1854, na obra *An Investigation of the Laws of Thought*.
- Em 1937, **Claude Shannon** aplicou-a ao projeto de circuitos digitais.
- Base matemática de todo hardware digital moderno.

Por que estudar?

É a fundação da lógica digital, dos processadores e da simplificação de código.

Operações Fundamentais

AND (Conjunção) — $A \cdot B$

Resultado **1** somente se **ambas** as entradas forem 1.

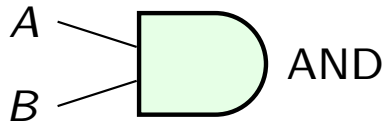
OR (Disjunção) — $A + B$

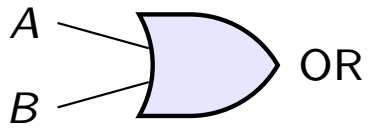
Resultado **1** se **ao menos uma** entrada for 1.

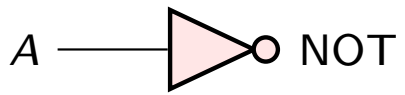
NOT (Negação) — $\overline{A}$

Inverte o valor: 0 vira 1 e 1 vira 0.

A	B	$A \cdot B$	$A + B$	$\overline{A}$
0	0	0	0	1
0	1	0	1	1
1	0	0	1	0
1	1	1	1	0

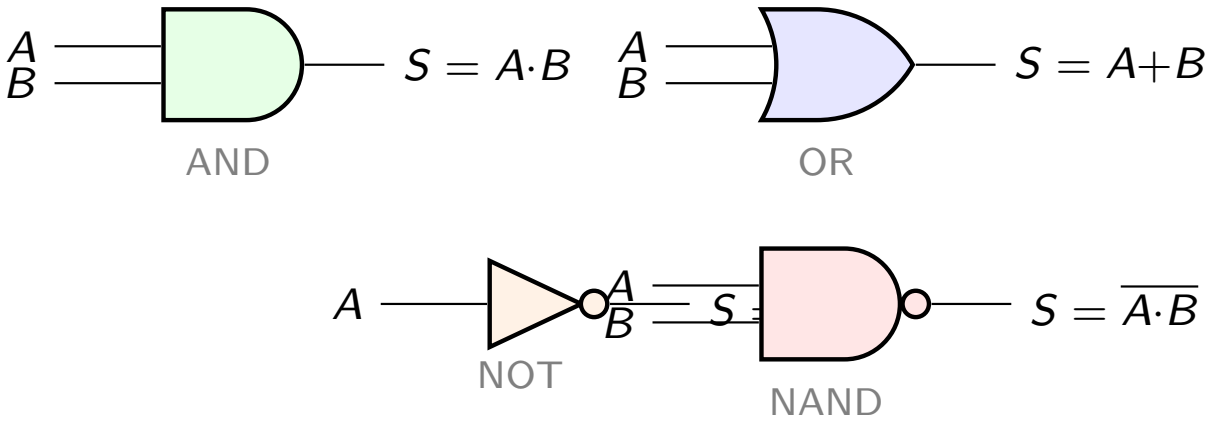






Portas Lógicas

Cada operação booleana corresponde a um componente eletrônico chamado **porta lógica**, implementado com transistores.



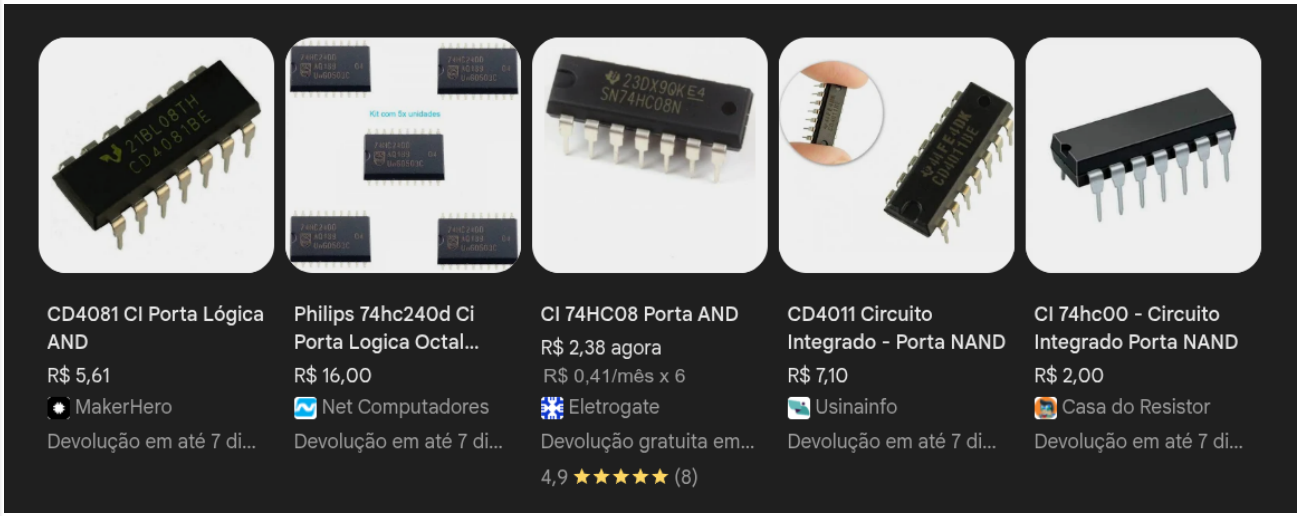
Transistores BJT e CMOS implementam essas portas em silício.

Componentes Físicos e Implementação Real

Hardware no "Mundo Real"

Na prática, as portas lógicas são encontradas em **Circuitos Integrados (CIs)**, como a famosa série 7400.

- **Encapsulamento:** Proteção física e pinagem padrão.
- **Miniaturização:** Bilhões de portas em um chip.
- **Sinais Elétricos:** 0 (GND) e 1 (V_{CC}).



Exemplo de CIs com portas lógicas discretas.

Aplicações Móveis e Industriais

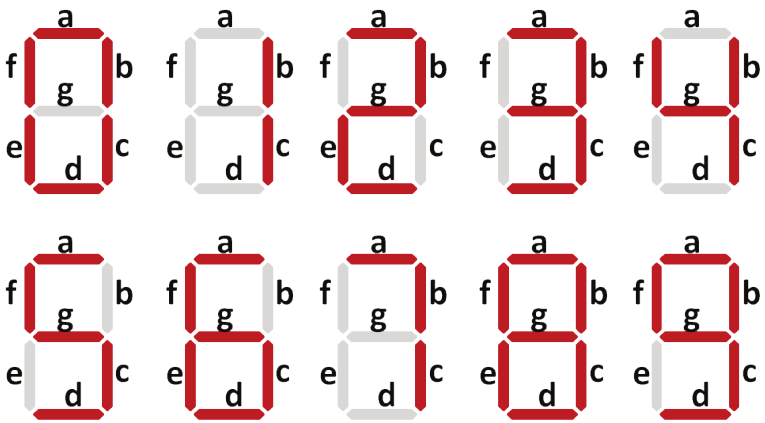
Desde o controle de um semáforo até a lógica de execução de um smartphone, tudo depende da orquestração física desses componentes.

Aplicação Prática: Display de 7 Segmentos

O que é?

Um componente eletrônico comum usado para exibir numerais decimais.

- Composto por 7 LEDs (segmentos de **a** até **g**).
- Cada segmento é controlado de forma independente.
- Um dos maiores exemplos de uso de **Lógica Combinacional**.

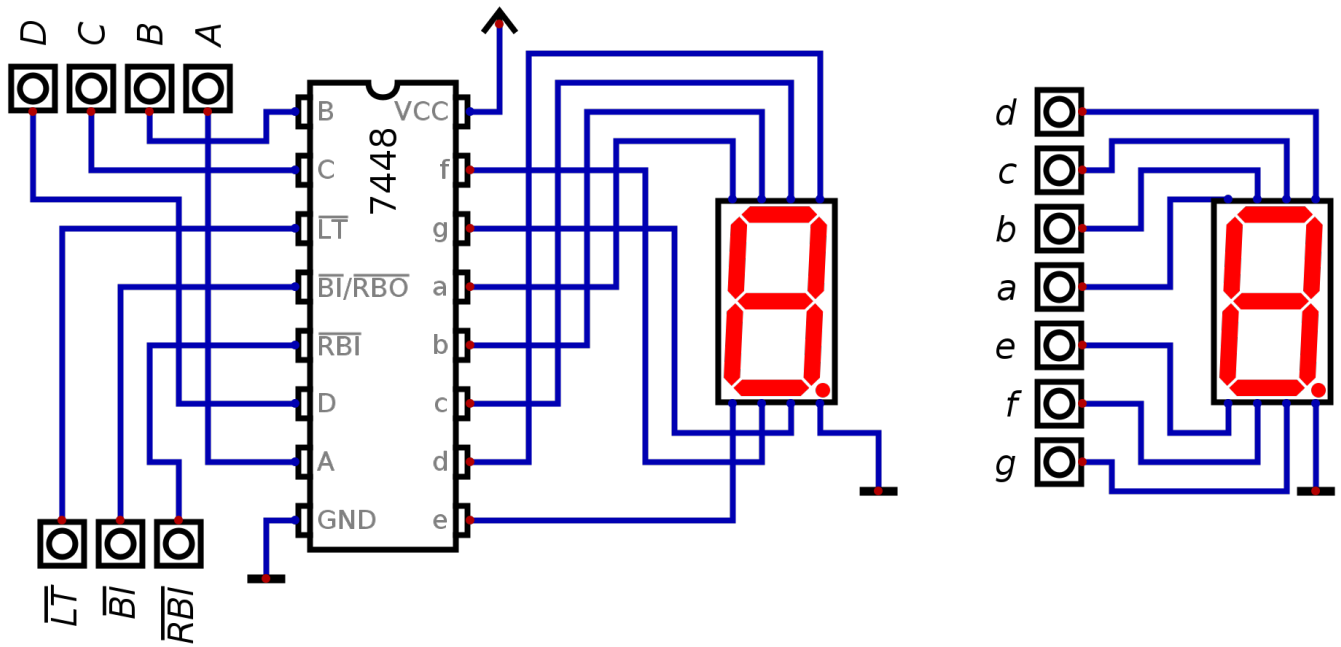


Estrutura de segmentos (a-g).

Como funciona a decodificação?

A Lógica por trás dos Números

Para cada número que queremos exibir, precisamos de uma **expressão booleana** específica para cada segmento.



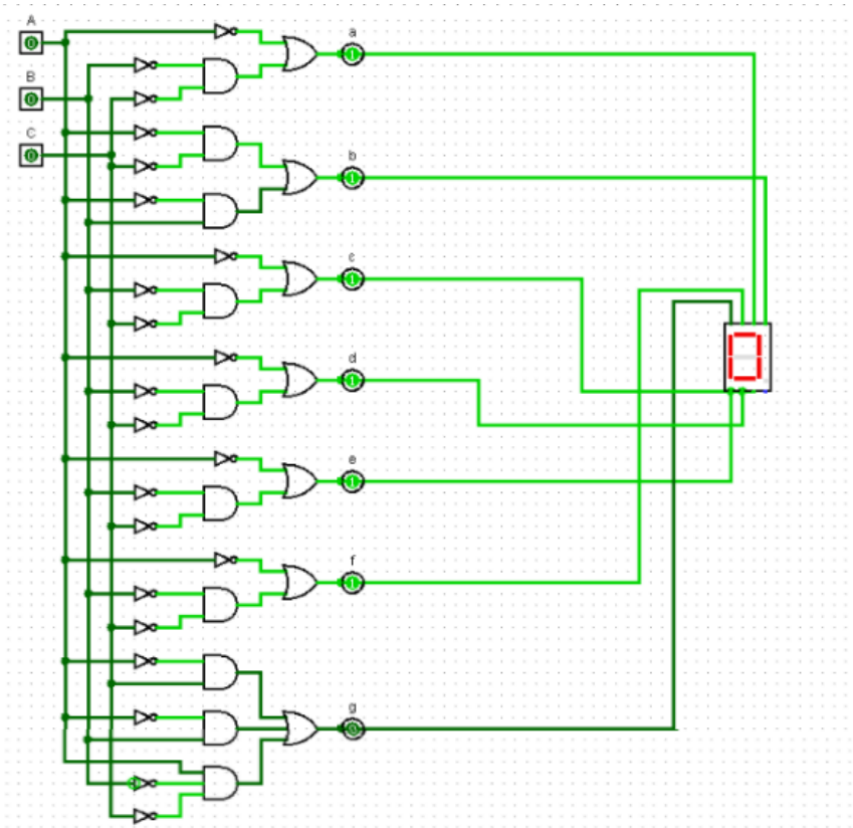
Exemplo: O número "1"

Para formar o número "1", os segmentos **b** e **c** devem ser 1 (LIGADOS), enquanto **a, d, e, f, g** devem ser 0 (DESLIGADOS).

Implementação com Portas Lógicas

O Decodificador de 7 Segmentos

Um circuito digital recebe um número em binário (ex: 4 bits) e ativa os segmentos correspondentes através de uma rede de portas lógicas.



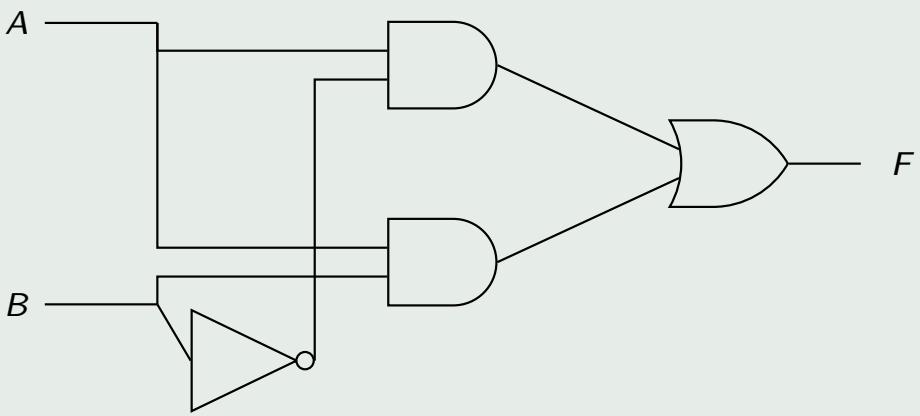
Nota: A simplificação booleana aqui é CRUCIAL para reduzir o número de transistores no chip.

Simplificação Algébrica

Exemplo

Simplificar $F = A\overline{B} + AB$

Circuito original



4 portas: 2 AND + 1 OR + 1 NOT

Circuito simplificado



Sem portas — conexão direta

Resultado

$$F = A$$

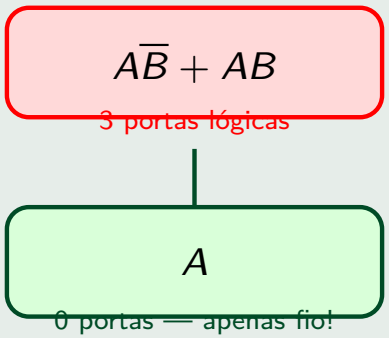
Simplificação Algébrica

Objetivo

Reduzir uma expressão booleana ao menor número de operações, economizando portas lógicas no hardware ou condicionais no código.

Exemplo: Simplificar $F = A\overline{B} + AB$

Circuito original vs. simplificado

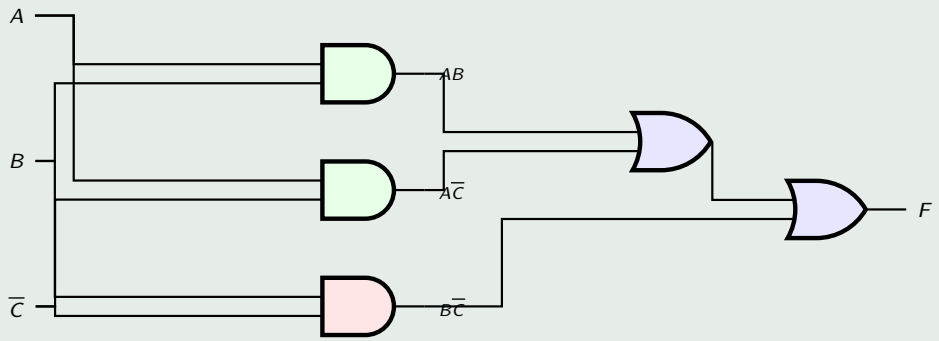


Simplificação de Circuito com 3 Variáveis

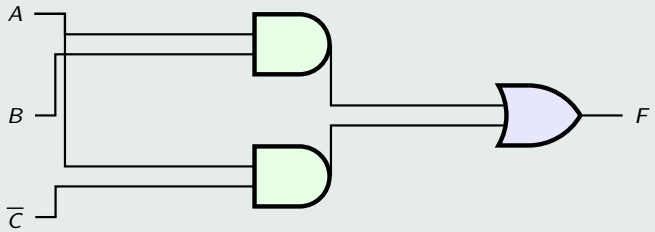
Problema

Simplificar a expressão: $F = AB + A\overline{C} + B\overline{C}$

Circuito original — 5 portas



Circuito simplificado — 3 portas



Redução: 5 portas → 3 portas (40%)

Simplificação de Condicionais em Código

Conexão direta

Toda condição lógica em código (`if`, `while`, `assert`) é uma **expressão booleana**. Simplificá-la reduz complexidade e elimina bugs.

Código redundante

```
if (autenticado == true
    && autenticado == true) {
    acessar();
}
```

Expressão: $A \cdot A = A$

Idempotência violada!

Código simplificado

```
if (autenticado) {
    acessar();
}
```

Expressão: A

Limpo e correto.

Outro exemplo — De Morgan

`!(A && B)` $\equiv$ `!A || !B` (Teorema de De Morgan)

Muito usado para inverter guardas em loops e simplificar condições de saída.

Refatoração de Lógica Complexa

Exemplo: Regra de Negócio para Desconto

Imagine um sistema onde um desconto é aplicado se o usuário é Administrador E Dono da conta E tem um Cupom, OU se ele é Administrador E NÃO é Dono E tem um Cupom.

Código sem simplificação

```
if ((isAdmin && isOwner && hasVoucher)
 || (isAdmin && !isOwner && hasVoucher)) {
    aplicarDesconto();
}
```

Aplicação da Álgebra

$A = \texttt{isAdmin}$

$B = \texttt{isOwner}$

$C = \texttt{hasVoucher}$

$$A \cdot B \cdot C + A \cdot \overline{B} \cdot C = A \cdot C \cdot (B + \overline{B})$$
$$= A \cdot C \cdot 1 = \boxed{A \cdot C}$$

Código Refatorado (Equivalente)

```
if (isAdmin && hasVoucher) {
    aplicarDesconto();
}
```

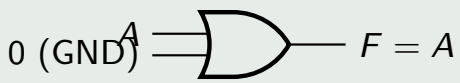
Resultado: A variável `isOwner` é irrelevante para o resultado final desta condição.

1. Leis de Identidade

Propriedade: Elemento Neutro

$$A + 0 = A \qquad \text{e} \qquad A \cdot 1 = A$$

Visão de Circuito (OR)



Simplificação: A porta OR é desnecessária; basta o fio de A.

Visão de Código (if)

```
// Inicial
if (A || false) { ... }

// Simplificado
if (A) { ... }
```

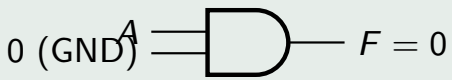
O valor constante não altera o resultado de A.

2. Leis de Nulidade

Propriedade: Elemento Dominante

$$A + 1 = 1 \qquad \text{e} \qquad A \cdot 0 = 0$$

Visão de Circuito (AND)



Simplificação: A saída é sempre 0, independente de A.

Visão de Código (if)

```
// Inicial
if (A && false) { ... }

// Simplificado
// (Código inacessível)
```

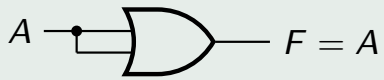
O compilador pode remover este bloco (*Dead Code*).

3. Leis de Idempotência

Propriedade: Repetição

$$A + A = A \qquad \text{e} \qquad A \cdot A = A$$

Visão de Circuito



Comparar o mesmo sinal consigo mesmo é redundante.

Visão de Código (if)

```
// Inicial
if (A || A) { ... }

// Simplificado
if (A) { ... }
```

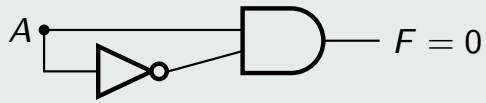
Ocorre frequentemente em lógicas de filtros complexos.

4. Leis do Complemento

Propriedade: Inversa

$$A + \overline{A} = 1 \qquad \text{e} \qquad A \cdot \overline{A} = 0$$

Visão de Circuito



Visão de Código (if)

```
// Inicial
if (A && !A) { ... }

// Simplificado
// Nunca executa
```

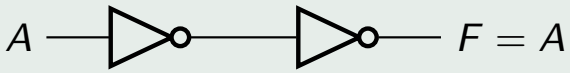
Uma variável não pode ser verdadeira e falsa ao mesmo tempo.

5. Lei da Dupla Negação

Propriedade: Involução

$$\overline{\overline{A}} = A$$

Visão de Circuito



Duas inversões seguidas retornam ao valor original.

Visão de Código (if)

```
// Inicial
if (!!A) { ... }

// Simplificado
if (A) { ... }
```

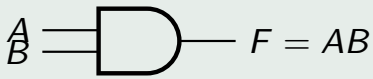
Comum em JavaScript para converter valores para booleano, mas logicamente redundante.

6. Leis Comutativas

Propriedade: Troca de Ordem

$$A + B = B + A \qquad \text{e} \qquad A \cdot B = B \cdot A$$

Visão de Circuito



A ordem das entradas não altera o resultado da porta lógica.

Visão de Código (if)

```
// Caso 1
if (A && B) { ... }

// Caso 2 (Equivalente)
if (B && A) { ... }
```

Nota: Em linguagens com *short-circuit*, a ordem pode afetar a performance.

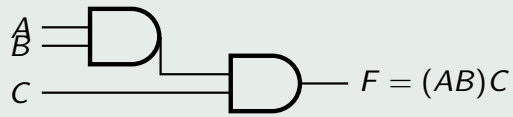
7. Leis Associativas

Propriedade: Agrupamento

$$(A + B) + C = A + (B + C)$$

$$(A \cdot B) \cdot C = A \cdot (B \cdot C)$$

Visão de Circuito (AND)



Visão de Código (if)

```
// Inicial
if ((A && B) && C) { ... }

// Simplificado
if (A && B && C) { ... }
```

Parentização excessiva pode ser removida para clareza.

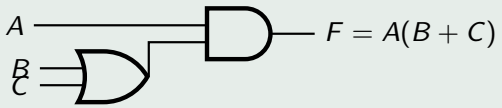
8. Lei Distributiva

Propriedade: Distribuição

$$A \cdot (B + C) = A \cdot B + A \cdot C$$

$$A + (B \cdot C) = (A + B) \cdot (A + C) \text{ (Única da Booleana!)}$$

Visão de Circuito



O circuito da direita economiza 1 porta lógica em relação ao da esquerda.

Visão de Código (if)

```
// Inicial
if (A && B || A && C) { ... }

// Simplificado
if (A && (B || C)) { ... }
```

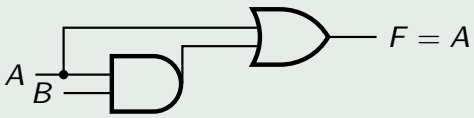
Fatorar variáveis comuns torna a lógica mais legível.

9. Lei da Absorção Simples

Propriedade: Redundância

$$A + (A \cdot B) = A \qquad \text{e} \qquad A \cdot (A + B) = A$$

Visão de Circuito



No hardware, economizamos uma porta AND e uma OR.

Visão de Código (if)

```
// Inicial
if (A || (A && B)) { ... }

// Simplificado
if (A) { ... }
```

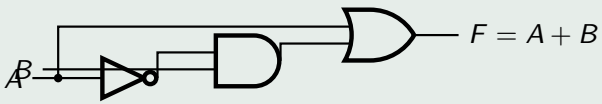
Se A já é verdadeiro, a segunda parte é irrelevante.

10. Absorção com Complemento

Propriedade: Simplificação Direta

$$A + (\overline{A} \cdot B) = A + B \qquad \text{e} \qquad A \cdot (\overline{A} + B) = A \cdot B$$

Visão de Circuito



Reduzimos de 3 portas para apenas 1 porta OR.

Visão de Código (if)

```
// Inicial
if (A || (!A && B)) { ... }

// Simplificado
if (A || B) { ... }
```

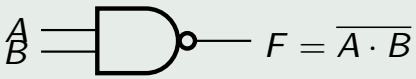
Uma construção comum que pode ser reduzida significativamente.

11. Primeiro Teorema de De Morgan

Propriedade: O "NÃO"de um produto

$$\overline{A \cdot B} = \overline{A} + \overline{B}$$

Visão de Circuito (NAND)



Uma porta NAND é equivalente a uma OR com entradas invertidas.

Visão de Código (if)

```
// Inicial
if (!(A && B)) { ... }

// Equivalente
if (!A || !B) { ... }
```

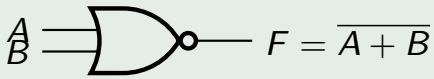
Fundamental para inverter condições de "saída antecipada".

12. Segundo Teorema de De Morgan

Propriedade: O "NÃO"de uma soma

$$\overline{A + B} = \overline{A} \cdot \overline{B}$$

Visão de Circuito (NOR)



Visão de Código (if)

```
// Inicial
if (!(A || B)) { ... }

// Equivalente
if (!A && !B) { ... }
```

"Se nem A nem B são verdadeiros"é o mesmo que "A é falso E B é falso".

Prova do Teorema do Consenso

Teorema

$$AB + \overline{A}C + BC = AB + \overline{A}C$$

Demonstração

$$F = AB + \overline{A}C + BC$$

$$= AB + \overline{A}C + BC(A + \overline{A}) \quad \text{(Identidade: } A + \overline{A} = 1\text{)}$$

$$= AB + \overline{A}C + ABC + \overline{A}BC \quad \text{(Distributiva)}$$

$$= (AB + ABC) + (\overline{A}C + \overline{A}BC)$$

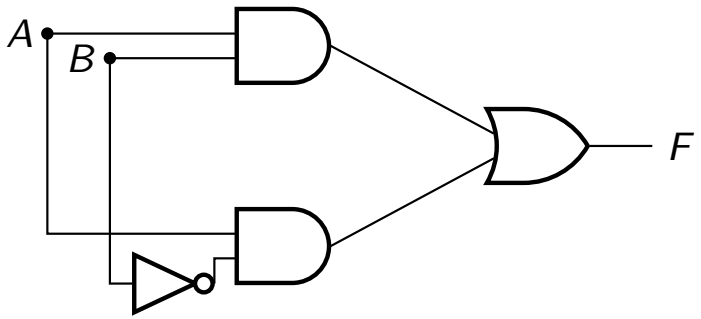
$$= AB(1 + C) + \overline{A}C(1 + B) \quad \text{(Fatoração)}$$

$$= AB + \overline{A}C \quad \text{(Lei: } 1 + X = 1\text{)}$$

Desafio 1: Simplificação de Circuito

Exercício

Simplifique a expressão booleana correspondente ao circuito abaixo e desenhe o circuito resultante:



Resposta 1: Simplificação Algébrica

Passo a Passo

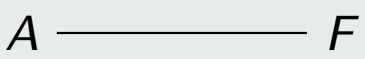
$$F = AB + A\overline{B}$$

$$F = A(B + \overline{B}) \quad \text{(Fatoração)}$$

$$F = A(1) \quad \text{(Complemento)}$$

$$\boxed{F = A}$$

Circuito Simplificado



Conclusão: O sinal de B não afeta o resultado final. O circuito é apenas um fio direto de A.

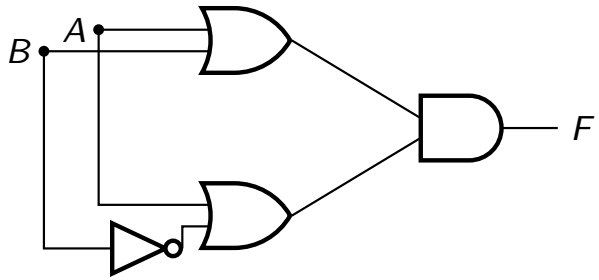
Desafio 2: Simplificação de Circuito

Exercício

Utilizando as propriedades da Álgebra Booleana, simplifique o circuito que implementa a seguinte função:

Expressão

$$F = (A + B) \cdot (A + \overline{B})$$



Resposta 2: Propriedade Distributiva

Passo a Passo

$$F = (A + B)(A + \overline{B})$$

$$F = A + (B \cdot \overline{B}) \quad \text{(Distributiva Inversa)}$$

$$F = A + 0 \quad \text{(Complemento)}$$

$$\boxed{F = A}$$

Explicação

Na Álgebra Booleana, a soma distribui sobre o produto: $X + (YZ) = (X + Y)(X + Z)$. Aqui fizemos o caminho inverso para

eliminar a variável B.

Desafio 3: Simplificação de Código

Exercício

Simplifique a estrutura condicional abaixo, removendo redundâncias lógicas:

Código Original

```
if ((idade >= 18 && temIngresso)
 || (idade >= 18 && !temIngresso)) {
    permitirEntrada();
}
```

Identifique as variáveis

- A: `idade >= 18`
- B: `temIngresso`

Resposta 3: Código Refatorado

Análise Lógica

Expressão: $A \cdot B + A \cdot \overline{B}$ Pela Lei da Absorção/Fatoração:
$A(B + \overline{B}) = A \cdot 1 = A$

Código Final

```
if (idade >= 18) {
    permitirEntrada();
}
```

Lição

Sempre que uma ação depende da mesma condição principal, independente de uma condição secundária ser verdadeira ou falsa, a condição secundária pode ser removida.

Desafio 4: Simplificação de Código

Exercício

Utilize o Teorema de De Morgan e a Dupla Negação para simplificar este `if`:

Código Original

```
if (!(!isAdmin || !temPermissao)) {
    abrirPainel();
}
```

Dica

Pense no Primeiro Teorema de De Morgan: $\overline{\overline{A} + \overline{B}}$

Resposta 4: Código Legível

Resolução

Expressão: $\overline{(\overline{A} + \overline{B})}$

1. Pelo Segundo Teorema de De Morgan: $\overline{\overline{A}} \cdot \overline{\overline{B}}$
2. Pela Dupla Negação: $A \cdot B$

Antes

```
if (!(!isAdmin || !temPermissao))
```

Depois

```
if (isAdmin && temPermissao)
```

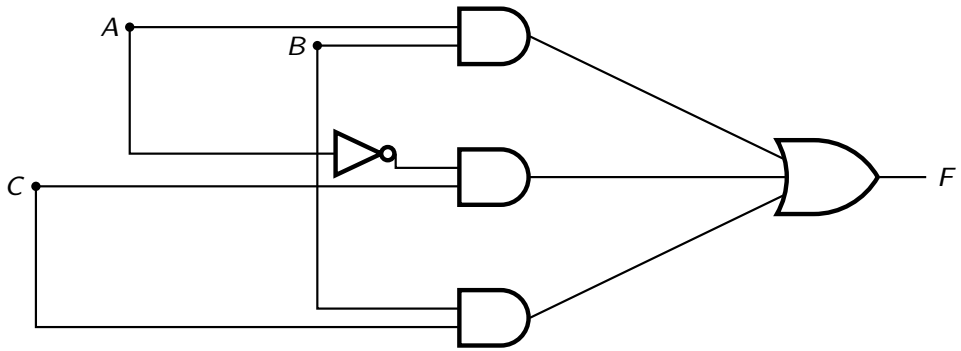
Dica de Carreira: Código que usa negações excessivas é mais difícil de manter. Prefira sempre a forma afirmativa simplificada.

Desafio 5: Teorema do Consenso (TikZ)

Exercício

Simplifique o circuito abaixo (Teorema do Consenso) e desenhe a versão reduzida:

$$F = AB + \overline{A}C + BC$$

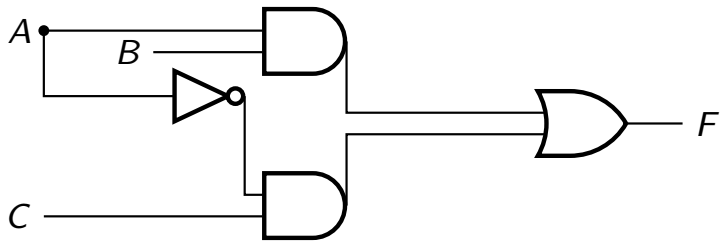


Resposta 5: Redundância Final

Simplificação

$$F = AB + \overline{A}C + BC \equiv AB + \overline{A}C$$

O termo BC não é necessário quando temos os "consensos"de A e $\overline{A}$.

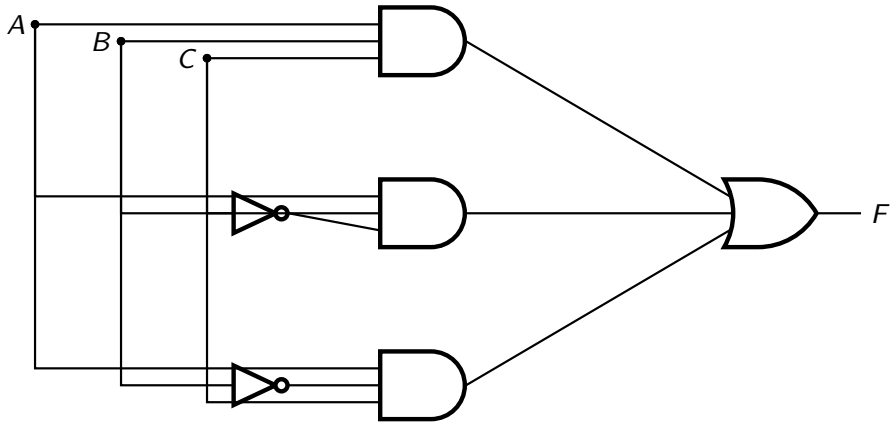


Desafio 6: Absorção Multivariável

Exercício

Simplifique a expressão booleana e o circuito correspondente:

$$F = ABC + AB\overline{C} + A\overline{B}C$$



Resposta 6: Simplificação Algébrica

Resolução

$$F = ABC + AB\overline{C} + A\overline{B}C$$

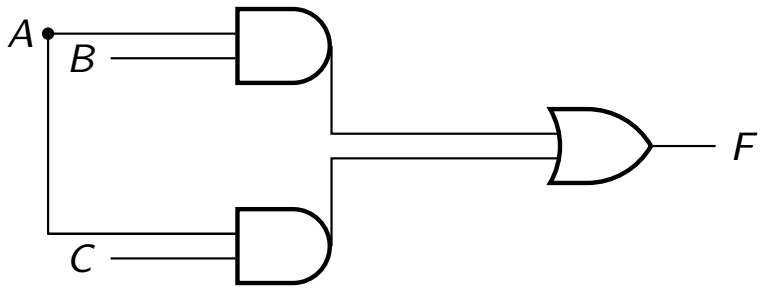
$$F = AB(C + \overline{C}) + A\overline{B}C \quad \text{(Fatoração)}$$

$$F = AB(1) + A\overline{B}C = AB + A\overline{B}C$$

$$F = A(B + \overline{B}C) \quad \text{(Fatoração novamente)}$$

$$F = A(B + C) \quad \text{(Lei da Absorção: } X + \overline{X}Y = X + Y\text{)}$$

$$\boxed{F = AB + AC}$$



Desafio 7: Lógica em Desenvolvimento

Exercício

Imagine que você herdou este código. Como você o simplificaria?

Código Original

```
if ((isAdmin && canEdit && isOwner)
 || (isAdmin && canEdit && !isOwner)) {
    unlockAsset();
}
```

Identificação

Variáveis: $A = \text{isAdmin}$, $B = \text{canEdit}$, $C = \text{isOwner}$. Expressão: $ABC + AB\overline{C}$

Resposta 7: Fatoração de Variáveis

Simplificação

$$ABC + AB\overline{C} = AB(C + \overline{C})$$

$$AB(1) = AB$$

Código Refatorado

```
if (isAdmin && canEdit) {
    unlockAsset();
}
```

Lição: Quando uma ação é permitida independentemente de uma variável ser true ou false, essa variável é irrelevante para a decisão.

Desafio 8: De Morgan e Absorção

Exercício

Simplifique a condição abaixo:

Código Original

```
if (!(!isValid || !isActive) || (isValid && !isActive)) {
    runProcess();
}
```

Dica

$A = isValid, B = isActive$. Expressão: $\overline{(\overline{A} + \overline{B})} + A\overline{B}$

Resposta 8: Resultado Limpo

Passo a Passo

1. De Morgan: $\overline{(\overline{A} + \overline{B})} \rightarrow A \cdot B$
2. Expressão completa: $AB + A\overline{B}$
3. Fatoração: $A(B + \overline{B}) = A \cdot 1 = A$

Código Final

```
if (isValid) {
    runProcess();
}
```

Por que simplificar?

Melhora a **legibilidade**, reduz a carga cognitiva do desenvolvedor e o número de testes unitários necessários.